Wamo Interview Prep — PF to OOP to DSA

Built from: 3 prior failures (Wamo x2, Contour x1, TBS x1) + Wamo's actual past questions (Number of Islands, Binary Tree Max Path Sum, Median from Data Stream).

The real gap is pattern recognition under pressure without AI help. Trace every code block below BY HAND before tomorrow. Reading isn't enough.

PART 1 — Programming Fundamentals (PF)

1.1 Pass by Value vs Pass by Reference

Value = function gets a COPY, changes don't affect original. Reference (&) = function gets the ACTUAL variable, changes persist.

```
void byValue(int x)  { x = 100; }      // caller's variable unchanged
void byRef(int &x)   { x = 100; }      // caller's variable changed
void byPtr(int *x)   { *x = 100; }     // changed via dereference

int a = 5;
byValue(a);  // a still 5
byRef(a);    // a now 100
byPtr(&a);   // a now 100
```

1.2 Stack vs Heap

Stack = automatic, fast, fixed size, freed when function returns (local vars). Heap = manual (new/delete), slow, dynamic size, YOU must free it or it leaks.

```
int x = 5;                // stack
int* p = new int(5);      // heap
delete p;                 // must free manually, else memory leak
```

1.3 Recursion — Base Case + Recursive Case

Every recursive function needs: (1) a base case that stops it, (2) a recursive case that moves toward the base case.

```
int factorial(int n) {
    if (n <= 1) return 1;       // base case
    return n * factorial(n-1);  // recursive case, n shrinks toward base
}
```

1.4 Arrays vs Pointers

Array name decays to a pointer to its first element. arr[i] is literally *(arr + i) — same thing, different syntax.

```
int arr[5] = {1,2,3,4,5};
int *p = arr;        // p points to arr[0]
cout << *(p+2);      // same as arr[2] -> prints 3
```

PART 2 — Object Oriented Programming (OOP)

2.1 The Four Pillars

Pillar	What it means	Real example
Encapsulation	Bundle data + methods, hide internals (private)	BankAccount hides balance, exposes deposit()/withdraw()
Abstraction	Show WHAT, hide HOW	Car: you press accelerator, don't see engine internals
Inheritance	Child class reuses parent's code	Dog and Cat both inherit from Animal

Polymorphism	Same function call, different behavior	shape->area() works differently for Circle/Square

2.2 Static vs Dynamic Binding

Static binding = decided at COMPILE time (function overloading, normal function calls). Dynamic binding = decided at RUN time (virtual functions / polymorphism).

```
class Animal {
public:
    virtual void speak() { cout << "Animal sound"; }   // virtual = dynamic binding
};
class Dog : public Animal {
public:
    void speak() override { cout << "Bark"; }
};

Animal* a = new Dog();
a->speak();  // prints "Bark" -> decided at RUNTIME because of virtual
```

Without 'virtual', this would print 'Animal sound' (static binding) — this exact trick is a common interview trap question.

2.3 Function Overloading vs Overriding

	Overloading	Overriding
Where	Same class	Parent/child class
Signature	Must differ (params)	Must be IDENTICAL
Binding	Compile-time (static)	Run-time (dynamic, needs virtual)

2.4 Abstract Class vs Interface (C++ has no 'interface' keyword)

```
class Shape {                          // abstract class
public:
    virtual double area() = 0;         // pure virtual -> makes class abstract
};
// Cannot do: Shape s;  -> compile error
// Must inherit and implement area()

class Circle : public Shape {
    double r;
public:
    Circle(double r) : r(r) {}
    double area() override { return 3.14 * r * r; }
};
```

A 'pure interface' in C++ = abstract class where EVERY method is pure virtual (no implementation at all).

PART 3 — Data Structures: Pattern Recognition

This is THE skill that decides pass/fail. When you read a problem, match it to a SHAPE below before writing any code.

Keyword in problem	Use this structure	Why
Connected regions / grid / islands	DFS or BFS on 2D grid	Need to explore all connected neighbors
Tree, root, paths, ancestors	Recursion (DFS on tree)	Tree problems almost always recurse top-down
Running median / top-K / 'kth largest'	Heap (priority_queue)	O(log n) insert, O(1) peek at min/max
Next greater/smaller element	Stack	LIFO matches 'look back until bigger found'
Sliding window / subarray / substring	Two pointers / window	Avoid O(n^2) by moving pointers, not restarting

Fast lookup by ID/key	HashMap (unordered_map)	O(1) average lookup
Shortest path, levels, layers	BFS with queue	BFS explores level by level = shortest path in unweighted graph
Sorted data + search	Binary search	O(log n) instead of O(n)
Undo/redo, matching brackets	Stack	Last-in-first-out matches 'undo last action'
Cycle detection in linked list	Fast/slow pointer (Floyd's)	Fast pointer laps slow pointer if cycle exists

3.1 DFS Template (memorize this shape, reuse everywhere)

```
void dfs(vector<vector<char>>& grid, int r, int c) {
    // 1. bounds check
    if (r < 0 || c < 0 || r >= grid.size() || c >= grid[0].size()) return;
    // 2. stop condition
    if (grid[r][c] == '0') return;
    // 3. mark visited
    grid[r][c] = '0';
    // 4. recurse in all directions
    dfs(grid, r+1, c); dfs(grid, r-1, c);
    dfs(grid, r, c+1); dfs(grid, r, c-1);
}
```

3.2 BFS Template

```
queue<pair<int,int>> q;
q.push({startR, startC});
visited[startR][startC] = true;
while (!q.empty()) {
    auto [r, c] = q.front(); q.pop();
    for (auto [dr, dc] : directions) {
        int nr = r+dr, nc = c+dc;
        if (valid(nr, nc) && !visited[nr][nc]) {
            visited[nr][nc] = true;
            q.push({nr, nc});
        }
    }
}
```

3.3 Two-Heap Pattern (running median)

```
priority_queue<int> maxHeap;                          // left half, largest on top
priority_queue<int, vector<int>, greater<int>> minHeap; // right half, smallest on top

void addNum(int num) {
    maxHeap.push(num);
    minHeap.push(maxHeap.top()); maxHeap.pop();
    if (minHeap.size() > maxHeap.size()) {
        maxHeap.push(minHeap.top()); minHeap.pop();
    }
}
double findMedian() {
    if (maxHeap.size() > minHeap.size()) return maxHeap.top();
    return (maxHeap.top() + minHeap.top()) / 2.0;
}
```

Logic: maxHeap always holds the SMALLER half, minHeap holds the LARGER half. Median is either top of bigger heap, or average of both tops.

3.4 Time Complexity Cheat Table

Structure	Access	Search	Insert	Delete
Array	O(1)	O(n)	O(n)	O(n)
Linked List	O(n)	O(n)	O(1)*	O(1)*

Stack/Queue	O(n)	O(n)	O(1)	O(1)
HashMap	-	O(1) avg	O(1) avg	O(1) avg
BST (balanced)	O(log n)	O(log n)	O(log n)	O(log n)
Heap	-	O(n)	O(log n)	O(log n)

*with pointer already at the node. (insert/delete at arbitrary position needs O(n) to find it first)

PART 4 — The 3 Wamo Questions, Re-derived

Don't memorize the code. Memorize the THOUGHT PROCESS — that's what they're testing when they change the wording next time.

4.1 Number of Islands

Trigger words: grid, connected, land/water -> 2D DFS/BFS. **Thought process:** loop every cell; on unvisited '1', that's a NEW island, sink the whole connected blob with DFS so you don't recount it; increment counter once per blob.

```
int numIslands(vector<vector<char>>& grid) {
    int count = 0;
    for (int i = 0; i < grid.size(); i++)
        for (int j = 0; j < grid[0].size(); j++)
            if (grid[i][j] == '1') { count++; dfs(grid, i, j); }
    return count;
}
```

4.2 Binary Tree Maximum Path Sum

Trigger words: tree, path, max/min sum -> recursive DFS returning a value upward. **Thought process:** for each node, the BEST path THROUGH it = node + best left branch (if positive) + best right branch (if positive). But what you RETURN to the parent can only be node + ONE side (a path can't branch twice).

```
int ans = INT_MIN;
int dfs(TreeNode* root) {
    if (!root) return 0;
    int left  = max(0, dfs(root->left));   // ignore negative branches
    int right = max(0, dfs(root->right));
    ans = max(ans, root->val + left + right);  // best path THROUGH this node
    return root->val + max(left, right);       // best path UP to parent (only one branch)
}
```

4.3 Find Median From Data Stream

Trigger words: running/continuous + median/top-K -> two heaps. **Thought process:** keep data split into two balanced halves; smaller half in a max-heap (so its largest is on top), larger half in a min-heap (so its smallest is on top); median sits exactly at the boundary between the two tops.

PART 5 — Tomorrow: How to Actually Perform

5.1 When you see a new problem

1. Read it twice. Identify the TRIGGER WORD from the table in Part 3.
2. Say your data structure choice OUT LOUD before coding: 'This looks like a graph problem, I'll use DFS because...'
3. Write the function signature first, then the base case, then the main loop.
4. Talk while you type — interviewers grade the PROCESS, not just the final answer.

5.2 If you get stuck

Do NOT go silent. Say: 'My first instinct is to brute-force with O(n^2) by checking every pair — let me think if there's a pattern to optimize.' This alone passes you in many interviews even with an imperfect final answer.

5.3 Known weak point — fix tonight

You've failed Wamo twice on the same style of question (graph/tree/heap pattern matching). Before sleeping, manually re-type all 3 code blocks in Part 4 WITHOUT looking, then check against this sheet. Do it twice.

PART 6 — CodeHedge Style: No If-Else Tricks

Pattern: 'classify X without conditional statements.' They want bitwise/modulo/ternary tricks instead of if/else.

6.1 Even/Odd without if-else

```
// Using modulo + array indexing
string labels[2] = {"Even", "Odd"};
cout << labels[n % 2];

// Using bitwise AND (fastest, classic trick)
cout << ((n & 1) ? "Odd" : "Even");   // ternary counts as no if-else in most rules
// if ternary is also banned:
string result[2] = {"Even", "Odd"};
cout << result[n & 1];
```

Why it works: n & 1 checks the last bit. 0 = even, 1 = odd. No branching instruction needed.

6.2 Palindrome check (string given, often paired with this round)

```
bool isPalindrome(string s) {
    int l = 0, r = s.size() - 1;
    while (l < r) {
        if (s[l] != s[r]) return false;   // this if is fine, it's not classification logic
        l++; r--;
    }
    return true;
}
// without any condition at all (pure recursion style):
bool isPalindromeRec(string s, int l, int r) {
    return (l >= r) || (s[l] == s[r] && isPalindromeRec(s, l+1, r-1));
}
```

6.3 Sign of a number without if-else

```
// returns -1, 0, or 1
int sign(int n) {
    return (n > 0) - (n < 0);   // bool subtraction trick
}
```

PART 7 — Recursive vs Iterative (always be ready for BOTH)

Interviewers commonly ask: 'now do it without recursion' or 'now do it without a loop.' Practice converting on the spot.

7.1 Factorial — both versions

```
// Recursive
int factR(int n) { return n <= 1 ? 1 : n * factR(n-1); }

// Iterative
int factI(int n) {
    int result = 1;
    for (int i = 2; i <= n; i++) result *= i;
    return result;
}
```

7.2 Reverse a Linked List — both versions

```
// Iterative (most commonly expected first)
ListNode* reverseIter(ListNode* head) {
    ListNode *prev = nullptr, *curr = head;
    while (curr) {
        ListNode* next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}

// Recursive
ListNode* reverseRec(ListNode* head) {
    if (!head || !head->next) return head;
    ListNode* newHead = reverseRec(head->next);
    head->next->next = head;
    head->next = nullptr;
    return newHead;
}
```

7.3 Fibonacci — both, plus the optimization they may ask for next

```
// Recursive (exponential, bad - O(2^n))
int fibR(int n) { return n <= 1 ? n : fibR(n-1) + fibR(n-2); }

// Iterative (O(n), what they actually want)
int fibI(int n) {
    if (n <= 1) return n;
    int a = 0, b = 1;
    for (int i = 2; i <= n; i++) { int c = a + b; a = b; b = c; }
    return b;
}
```

If you give the recursive Fibonacci, expect an immediate follow-up: 'what's the time complexity, can you optimize it?' Answer: O(2^n) -> fix with memoization or iterative O(n).

PART 8 — DiveBridge Style: API / Query Optimization

This round isn't DSA — it's system-thinking. They ask: given a slow API/query, how do you fix it. Have a structured 3-layer answer ready.

8.1 The structured answer (say this out loud, in order)

Layer	Fix	Example
Database	Add index, avoid N+1 queries	select_related()/prefetch_related() in Django, or proper JOIN instead of loop queries
Application	Cache repeated reads	Redis cache with TTL on frequently-read, rarely-changed data
Infrastructure	Load balance / paginate	Don't return 10,000 rows at once; paginate; use a load balancer for traffic spikes

8.2 N+1 query problem (the single most asked backend bug)

Problem: fetching a list, then looping to fetch related data per item = 1 + N queries instead of 1.

```
# BAD - N+1 problem (1 query for users, then N queries for orders)
users = User.objects.all()
for u in users:
    print(u.orders.all())     # hits DB every single loop

# GOOD - one query with JOIN
users = User.objects.prefetch_related('orders').all()
```

```
for u in users:
    print(u.orders.all())     # already loaded, no extra query
```

8.3 Redux/state management one-liner (if they bring up frontend state)

Redux centralizes state outside components so multiple components can read/update the same data without prop-drilling. Flow: Component dispatches an ACTION -> REDUCER updates STATE -> STORE notifies subscribed components -> UI re-renders.

PART 9 — Rapid Drill List (do these tonight, time yourself)

Solve each in under 10 minutes WITHOUT looking at notes, then check. This is the actual fix for AI-dependence — repetition without help.

1. Reverse a string (iterative + recursive)
2. Check if a number is prime
3. Find the second largest element in an array
4. Remove duplicates from an array
5. Two Sum (array + target, return indices)
6. Valid Parentheses (stack)
7. Reverse a Linked List (iterative + recursive)
8. Detect cycle in Linked List (fast/slow pointer)
9. Number of Islands (DFS on grid)
10. Binary Tree Inorder/Preorder/Postorder traversal
11. Maximum Depth of Binary Tree
12. Kth Largest Element (heap)
13. Merge Two Sorted Lists
14. Binary Search (iterative)
15. FizzBuzz without if-else (modulo + string building)

Final note

You are not weak at logic — you're under-practiced at doing it WITHOUT a tool reformulating it for you. Every pattern above repeats across interviews with different wrapping. Recognize the shape, not the wording.